RAPPORT DE PROJET

Développement d'un CRM Web Full SaaS

Architecture Cloud Moderne

Projet	Année universitaire
AAAS CRM	**2025 — 2026**
Auteur	Formation
Alim Samira	**Communication Digitale**

Stack technique : Next.js 14 · Supabase · Brevo · Vercel · GitHub

1. Introduction et contexte du projet

Dans le cadre de la formation en Communication Digitale, ce projet a pour objectif la conception et le développement d'une application web de type CRM (Customer Relationship Management) en mode Full SaaS. L'entreprise fictive AAAS souhaitait centraliser la gestion de ses relations clients, automatiser ses communications marketing et disposer d'un tableau de bord analytique pour piloter ses performances commerciales.

Ce rapport présente l'ensemble du travail réalisé : analyse des besoins, modélisation de la base de données, architecture technique retenue, fonctionnalités développées, ainsi que les choix technologiques effectués.

Objectifs pédagogiques atteints

- Concevoir et déployer une application web full-stack (frontend + backend + base de données)
- Intégrer des services SaaS et APIs externes (Supabase, Brevo, Vercel)
- Développer une interface responsive moderne avec Next.js 14 et Tailwind CSS
- Mettre en place un système d'authentification et d'autorisation avec RBAC (4 rôles)
- Automatiser des campagnes d'emailing et analyser leur performance (taux d'ouverture, clics)
- Déployer en production avec CI/CD via Vercel relié à GitHub
- Modéliser une base de données relationnelle avec PostgreSQL via Supabase

2. Analyse des besoins

L'analyse des besoins a permis d'identifier les acteurs du système et leurs interactions avec l'application, ainsi que les fonctionnalités indispensables à couvrir.

2.1 Acteurs identifiés

Rôle	Description	Droits principaux
Master Admin	alim.samira2002@gmail.com — compte protégé	Accès total, gestion des rôles, campagnes
Admin	Utilisateur avec rôle admin approuvé	Gestion complète des modules
Commercial	Membre de l'équipe commerciale	Pipeline, leads, contacts, tâches
Partenaire	Partenaire externe	Accès restreint selon permissions
Utilisateur Standard	Employé ou collaborateur	Accès limité selon permissions admin

2.2 Fonctionnalités attendues et couverture

Critère	Poids	Statut
Gestion des utilisateurs (auth + rôles)	—	Implémenté
Gestion des contacts	—	Implémenté
Gestion des entreprises	—	Implémenté
Gestion des leads	—	Implémenté
Pipeline et funnel de vente (Kanban)	—	Implémenté
Gestion des tâches avec assignation	—	Implémenté
Automatisation emailing (Brevo)	—	Implémenté
Tableau de bord analytique + KPIs	—	Implémenté
Notifications temps réel	—	Implémenté
Design responsive (mobile + desktop)	—	Implémenté
Déploiement CI/CD (Vercel + GitHub)	—	Implémenté
Containerisation Docker	—	

3. Modélisation des données

La modélisation de la base de données du projet AAAS CRM a suivi une démarche progressive en trois étapes. Le processus a débuté par une esquisse initiale sous forme de diagramme de classes UML et d'un premier diagramme ER, permettant d'identifier rapidement les entités principales et leurs relations. Ces représentations préliminaires ont ensuite guidé la conception du MCD (Modèle Conceptuel de Données), qui constitue la modélisation complète et définitive de l'ensemble du système.

3.1 MCD — Modèle Conceptuel de Données

Le MCD est la modélisation conceptuelle finale et complète du projet. Il intègre l'ensemble des 10 entités identifiées au cours de la conception : PROFILES, COMPANIES, CONTACTS, LEADS, TASKS, NOTIFICATIONS, EMAIL_CAMPAIGNS, ADMIN_PERMISSIONS, ACCESS_REQUESTS et INTERACTIONS. Contrairement aux diagrammes préliminaires, il représente avec précision toutes les relations entre entités, les cardinalités et les attributs clés de chaque table.

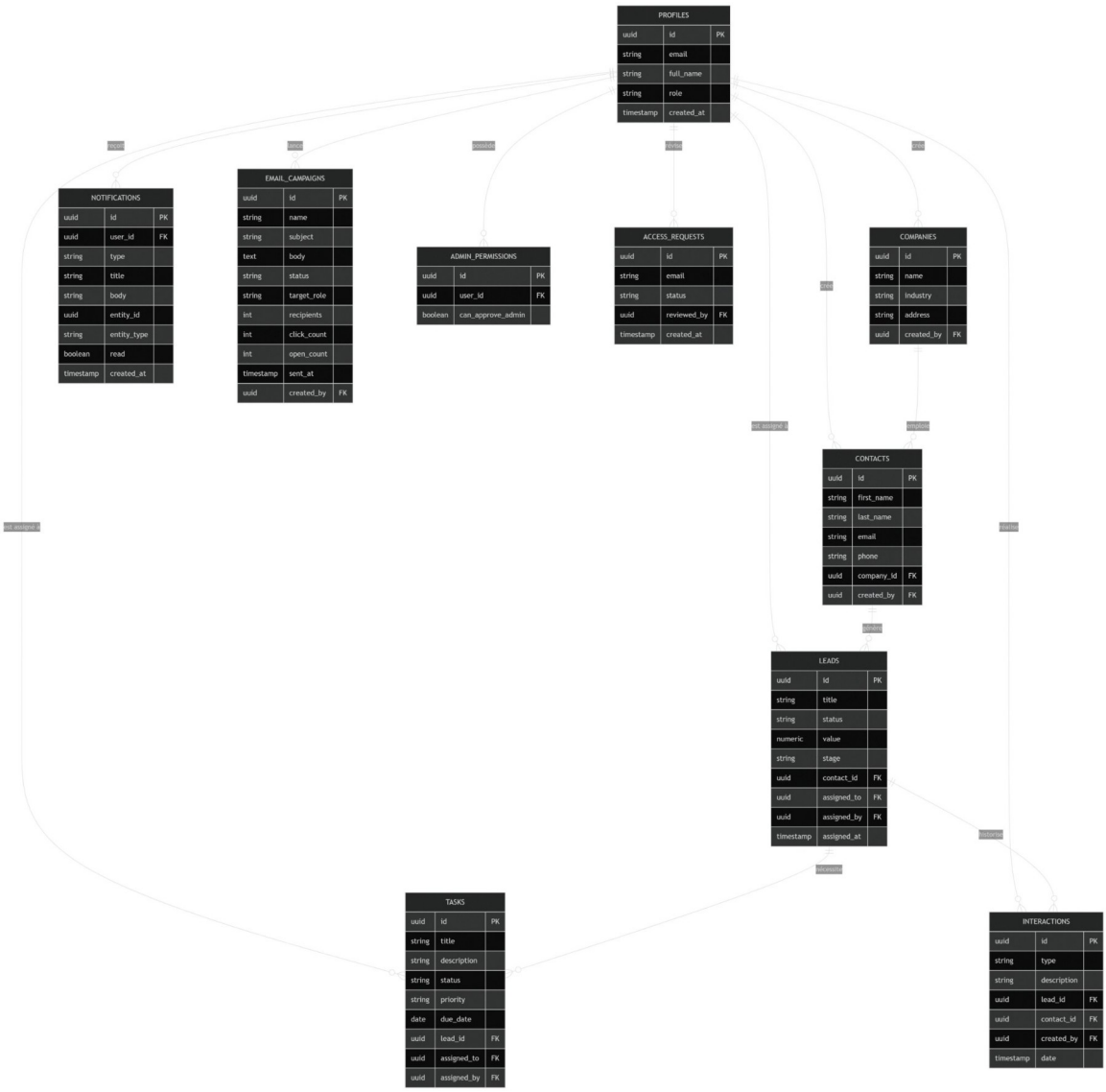

Figure 1 — MCD du projet AAAS CRM (entités et associations principales)

Les principales associations du MCD finalisé sont les suivantes :

- PROFILES → LEADS : un utilisateur peut se voir assigner plusieurs leads (assigned_to FK) et en créer plusieurs (assigned_by FK)
- PROFILES → TASKS : un utilisateur peut être assigné à plusieurs tâches et en créer d'autres (assigned_to / assigned_by)
- CONTACTS → LEADS : un contact peut générer plusieurs leads (contact_id FK) — le lead hérite du contexte commercial du contact
- COMPANIES → CONTACTS : une entreprise emploie plusieurs contacts (company_id FK) — permet de regrouper les interlocuteurs d'un même compte
- PROFILES → EMAIL_CAMPAIGNS : un administrateur crée et envoie des campagnes ciblées par rôle (created_by FK, target_role), avec suivi des ouvertures et clics
- LEADS → INTERACTIONS : un lead historise plusieurs interactions (appels, emails, réunions) via lead_id et contact_id FK PROFILES → NOTIFICATIONS : chaque utilisateur reçoit des notifications en temps réel (user_id FK) lors d'une assignation de tâche ou de lead PROFILES → ADMIN_PERMISSIONS : les permissions modulaires de chaque administrateur sont stockées (user_id FK, can_approve_admin)

3.2 Diagramme de Classes UML — Conception préliminaire

Le diagramme de classes UML a été produit en début de projet, lors de la phase d'analyse des besoins. À ce stade, l'objectif était d'identifier rapidement les grandes entités métier, leurs attributs principaux et les opérations associées (ex : Lead.assignTo(), EmailCampaign.send()), sans encore figer tous les détails techniques. Ce diagramme a joué le rôle d'outil de communication pour le projet et a directement alimenté la réflexion qui a conduit au MCD complet.

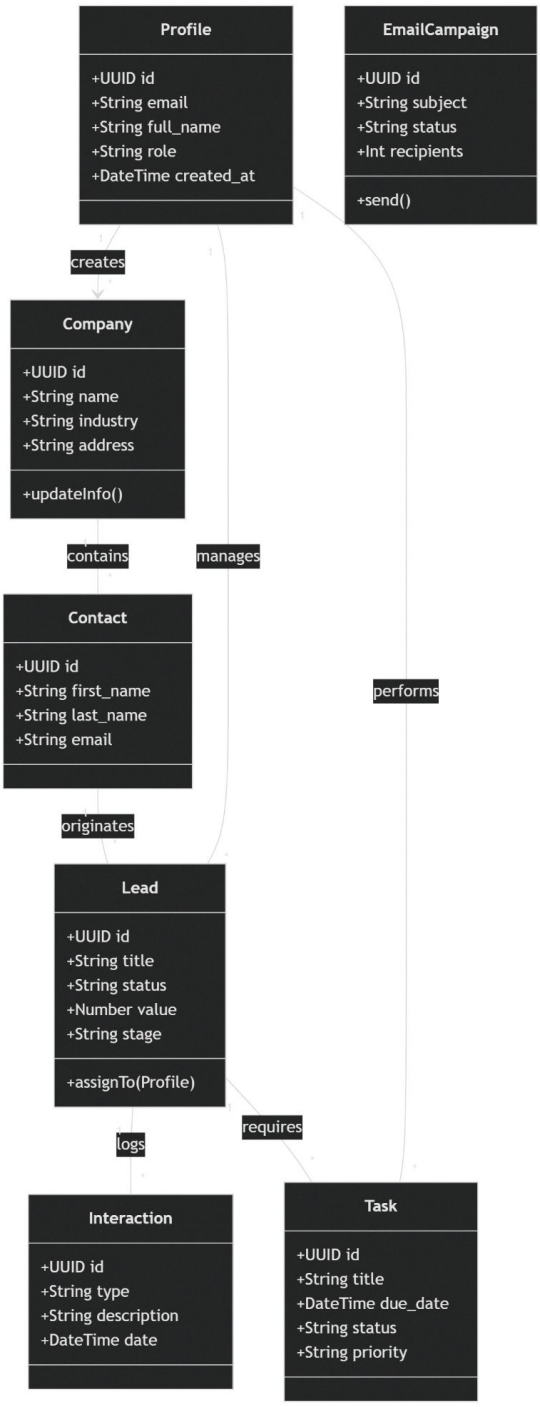

Figure 2 — Diagramme de Classes UML (attributs, types et méthodes)

Ce diagramme préliminaire couvre les 7 entités initiales identifiées (Profile, Company, Contact, Lead, Task, Interaction, EmailCampaign). Les entités NOTIFICATIONS, ADMIN_PERMISSIONS et ACCESS_REQUESTS ont été ajoutées dans un second temps, lors de la conception détaillée du MCD, après analyse des besoins en matière de sécurité, de RBAC et de notifications temps réel.

3.3 Diagramme Entité-Relation (ER) — Schéma intermédiaire

Le diagramme ER a été réalisé lors de la phase de prototypage de la base de données, avant la mise en production sur Supabase. Il représente le schéma intermédiaire, focalisé sur les tables les

plus actives du CRM (profiles, contacts, companies, leads, tasks, interactions). Ce diagramme a permis de valider les types de colonnes, les clés primaires (PK) et les clés étrangères (FK) avant implémentation, servant de pont entre la modélisation conceptuelle du MCD et le schéma SQL final.

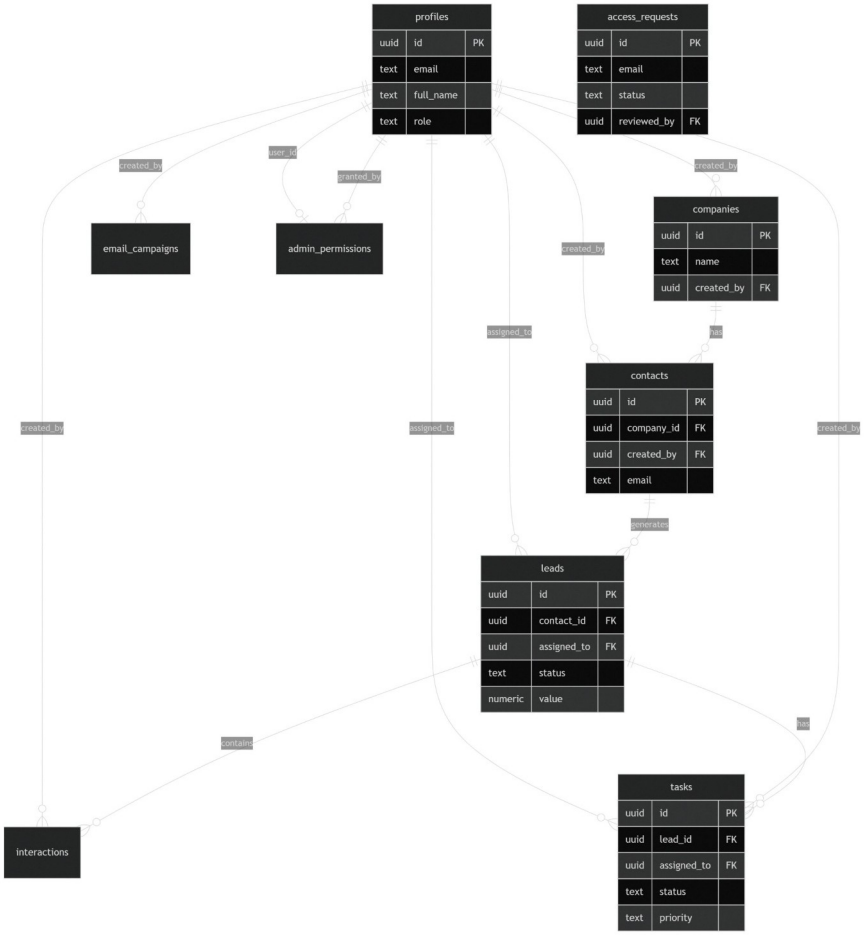

Figure 3 — Diagramme ER (schéma physique PostgreSQL / Supabase)

Ce diagramme intermédiaire reflète l'état de la base à la fin de la phase de prototype. Il a ensuite été enrichi lors de l'implémentation finale : les tables notifications, email_campaigns et admin_permissions, ainsi que les colonnes assigned_to, assigned_by et assigned_at sur leads et tasks, ont été ajoutées pour répondre aux exigences fonctionnelles complètes du CRM, telles que modélisées dans le MCD définitif.

4. Architecture technique

L'application adopte une architecture Full SaaS sans serveur à gérer. Chaque composant est géré par une plateforme cloud spécialisée, ce qui garantit scalabilité, sécurité et déploiement continu.

4.1 Stack technologique

Couche	Technologie	Rôle
Frontend	Next.js 14 + React + TypeScript	Interface utilisateur, routing, rendu SSR/CSR
Styling	Tailwind CSS + Glassmorphism	Design responsive, thème sombre, glassmorphisme
Backend	Next.js API Routes (serverless)	Endpoints REST sans serveur dédié
Base de données	Supabase (PostgreSQL)	BaaS : DB cloud + Auth + Realtime + RLS
Authentification	Supabase Auth (JWT)	Login/Logout, JWT, RBAC 4 rôles
Emailing	Brevo SMTP API	Campagnes email, tracking ouvertures et clics
Hébergement	Vercel	Déploiement automatique, CDN global, serverless
CI/CD	GitHub → Vercel	Push → build → deploy automatique
Variables d'env	Vercel Environment Variables	Clés API sécurisées hors du code source

4.2 Architecture des flux de données

Le flux de données suit le schéma suivant :

```
Navigateur (React/Next.js) → API Routes (serverless Vercel) → Supabase PostgreSQL
    ↕ Realtime WebSocket (notifications instantanées)
API Routes → Brevo SMTP API → Emails destinataires
```

- Le frontend communique avec Supabase directement via le client JS (anon key) pour les opérations CRUD protégées par RLS
- Les API routes serverless utilisent la service_role key pour les opérations administratives (création d'utilisateurs, envoi de campagnes)
- Supabase Realtime envoie les notifications en temps réel via WebSocket sans polling
- Brevo reçoit les emails via son API REST et retourne les events de tracking (ouvertures, clics) via webhook

4.3 Sécurité et Row Level Security (RLS)

La sécurité est assurée à deux niveaux :

- Niveau application : RBAC (Role-Based Access Control) avec 4 rôles — le rôle est lu depuis le JWT Supabase à chaque requête
- Niveau base de données : Row Level Security (RLS) — Supabase vérifie que chaque SELECT/INSERT/UPDATE ne porte que sur les données autorisées pour l'utilisateur courant (auth.uid())
- Master Admin protégé : un trigger SQL empêche toute modification du rôle du compte maître
- Mot de passe hashé et JWT signés par Supabase Auth — aucun mot de passe stocké en clair

5. Description des fonctionnalités

5.1 Authentification et gestion des rôles

- **Login / Logout via Supabase Auth (JWT)**
- Inscription avec validation email
- 4 rôles : Master Admin, Admin, Commercial, Partenaire, Utilisateur Standard
- Demande d'accès admin avec workflow d'approbation
- Permissions modulaires par rôle configurables par l'admin

5.2 Gestion des contacts

- **Création, modification, suppression de contacts**
- Fiches clients : nom, email, téléphone, entreprise associée
- Recherche et filtrage en temps réel
- Association automatique à une entreprise

5.3 Gestion des entreprises

- **CRUD complet sur les entreprises partenaires**
- Secteur d'activité, adresse, contacts associés
- Vue liste avec filtres

5.4 Pipeline Kanban (Gestion des leads)

- **Kanban visuel avec 6 colonnes : Nouveaux → Qualifiés → En cours → Proposition → Négociation → Convertis/Perdus**
- Drag & Drop entre colonnes (react-beautiful-dnd)
- Attribution de leads à un commercial avec notification temps réel
- Valeur financière du lead, contact lié, historique d'interactions

5.5 Gestion des tâches

- **Création de tâches avec priorité (Basse/Moyenne/Haute) et date d'échéance**
- Attribution à un utilisateur avec notification instantanée
- Statuts : À faire / En cours / Terminée / En retard
- Modification du statut directement depuis le tiroir de notifications
- Bouton "Demander de l'aide" qui notifie l'assigneur

5.6 Système de notifications temps réel

- **Cloche 🔔 dans la sidebar avec badge de comptage des non-lus**
- Tiroir de notifications (Bottom Sheet sur mobile, panneau latéral sur desktop)
- Notifications pour : tâches assignées, leads assignés, demandes d'aide
- Marquer comme lu, tout lire, supprimer individuellemnt
- Realtime via Supabase WebSocket + polling 30s de secours

5.7 Campagnes email (Brevo)

- **Création de campagnes avec sujet, corps, ciblage par rôle**
- Envoi individuel par email avec personnalisation (prénom, nom, email)
- Tracking des ouvertures et clics via pixel et webhook
- Historique des campagnes envoyées avec statistiques

5.8 Tableau de bord analytique

- **KPIs en temps réel : leads actifs, taux de conversion, tâches en retard**
- Graphiques de répartition : leads par statut, performance commerciale
- Statistiques de campagnes email : envois, ouvertures, clics
- Activité récente et alertes

6. Déploiement et DevOps

Le déploiement est entièrement automatisé grâce à la connexion GitHub → Vercel. Chaque push sur la branche main déclenche un nouveau build et déploiement en production.

6.1 Workflow CI/CD

- **Code versionné sur GitHub : https://github.com/Alim-Samira/aaas-crm**
- Vercel détecte chaque commit sur main et lance le build Next.js
- Variables d'environnement sécurisées dans Vercel (SUPABASE_URL, SUPABASE_ANON_KEY, BREVO_API_KEY…)
- URL de production : https://aaas-crm.vercel.app
- Déploiements de preview automatiques pour chaque Pull Request

6.2 Variables d'environnement requises

Variable	Description
NEXT_PUBLIC_SUPABASE_URL	URL du projet Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Clé anonyme Supabase (côté client)
SUPABASE_SERVICE_ROLE_KEY	Clé service_role (opérations admin)
BREVO_API_KEY	Clé API Brevo pour l'emailing
BREVO_SENDER_EMAIL	Email expéditeur vérifié dans Brevo
NEXT_PUBLIC_APP_URL	URL publique de l'application

7. Critères d'évaluation et conformité à l'énoncé

Le tableau suivant présente l'état de conformité du projet par rapport aux critères d'évaluation définis dans l'énoncé.

Critère	Poids	Statut
Fonctionnalités implémentées + Présentation orale	40%	Tous les modules requis
Qualité du code / bonnes pratiques	20%	TypeScript strict, API routes
UI / UX	15%	Glassmorphisme, responsive
Déploiement et CI/CD	15%	GitHub → Vercel automatique
Documentation	10%	README + diagrammes UML
Originalité du workflow pipeline	Bonus 5–10%	Kanban 6 étapes personnalisé

7.1 Points forts du projet

- Architecture Full SaaS complète sans serveur à administrer — exactement comme préconisé dans l'énoncé
- RBAC 4 rôles avec permissions modulaires configurables par l'admin en temps réel
- Pipeline Kanban personnalisé avec drag & drop et 6 étapes de vente définies par l'équipe
- Système de notifications temps réel Supabase (WebSocket natif) — fonctionnalité avancée au-delà du minimum requis
- Tracking complet des campagnes email (ouvertures + clics via pixel + webhook Brevo)
- Design glassmorphisme cohérent, thème sombre professionnel, 100% responsive mobile
- Sécurité renforcée : RLS PostgreSQL + JWT + triggers SQL de protection du Master Admin

7.2 Conformité aux exigences techniques

- Application web responsive — mobile-first avec barre de navigation bottom sur mobile
- Base de données relationnelle PostgreSQL — 10 tables, FK, contraintes, triggers
- Authentification JWT — Supabase Auth, mot de passe hashé, sessions sécurisées
- Interface ergonomique — glassmorphisme, animations, composants accessibles
- Bonnes pratiques de sécurité — RLS, variables d'env, no secret en frontend
- CI/CD — GitHub → Vercel, déploiements automatiques à chaque commit

8. Conclusion

Ce projet de CRM Full SaaS répond à l'ensemble des exigences définies dans l'énoncé. Il dépasse même le minimum requis sur plusieurs aspects : système de notifications temps réel, tracking complet des campagnes email, RBAC avancé avec permissions modulaires, et interface mobile-first.

Les compétences mobilisées couvrent l'intégralité du spectre demandé : développement frontend (React/Next.js), backend serverless (API Routes), base de données relationnelle (PostgreSQL/Supabase), intégration d'un service d'emailing (Brevo), déploiement cloud (Vercel) et gestion de versions (Git/GitHub).

L'architecture Full SaaS choisie démontre une maîtrise des pratiques modernes du développement web : aucun serveur à administrer, scalabilité immédiate, coûts initiaux faibles et sécurité gérée par les plateformes spécialisées.

— Alim Samira · Formation Communication Digitale · 2025–2026 —